

Introducción a PHP



Lic. Mariano Ingerto

marianoingerto@gmail.com

Estructuras Repetitivas (bucle For)

El bucle FOR se utiliza para repetir mas instrucciones un determinado número de veces.

De entre todos los bucles, el FOR se suele utilizar cuando sabemos seguro el número de veces que queremos que se ejecute la sentencia.

```
For ([Inicio de la variable];  
[Condición];  
[Incremento o decremento de la  
variable]) {  
  
[Instrucciones];  
  
}
```



Estructuras Repetitivas (bucle For)

```
<?php  
  
for ($f=1;$f<=100;$f++) {  
  
    echo $f ;  
  
}  
  
?>
```



Estructuras Repetitivas (bucle While)

El bloque se repite mientras la condición del **while** sea verdadera.

La condición del **while** se verifica antes de ingresar al bloque a repetir. Si la misma se verifica falsa la primera vez no se ejecutará el bloque.

```
while (condición)
```

```
{
```

```
[Instrucciones];
```

```
}
```



Estructuras Repetitivas (bucle While)

Ejemplo:

Generar un valor aleatorio entre 1 y 100, luego imprimir en la página desde 1 hasta el valor generado (de uno en uno)

```
<?php  
  
$valor=rand(1,100);  
  
$inicio=1;  
  
while ($inicio<=$valor) {  
  
    echo $inicio;  
  
    echo "<br>";  
  
    $inicio++; }  
  
?>
```



Rand: Genera un entero aleatorio (min/max)

Funciones en PHP

Una función consiste en un conjunto de rutinas y acciones que a lo largo del script van a ser ejecutadas multitud de veces agrupados en una **FUNCION** y desde cualquier punto del script puede ser llamada y ejecutada.

A su vez, esta función puede recibir parámetros externos de los cuales dependa el resultado de una función.

Las funciones deben ser colocadas siempre antes de realizar la llamada a la función (como es lógico). La sintaxis de una función es la siguiente:

```
function nombre(parámetros){  
    instrucciones de la función  
}
```

para llamar a la función sería de la siguiente forma:

```
nombre(parámetros)
```



Funciones Date

La función **date()** se usa para expresar fechas y horas en formato estándar GMT, y entre los paréntesis podemos agregar distintos comandos para que nos muestre un determinado formato y orden.

Código	Resultado
a	am o pm
A	AM o PM
h	Hora (de 1 a 12)
H	Hora (de 0 a 23)
i	Minutos
s	Segundos
j	Día (sin los ceros)
d	Día (con los ceros)
D	Abreviacion del día
l	Nombre del día
z	Numero de día del año (desde 1 hasta 365)
m	Numero del Mes
M	Abreviacion del mes
F	Nombre del mes
y	Año (formato de 2 digitos)
Y	Año (formato de 4 digitos)



Links con variables

- Vamos a la primera de las formas de "definirle un valor" o "setear" una variable: **los links.**
- Cuando en HTML hacemos un link, vemos lo siguiente:

```
<html>
<head>
<title>Paso de variables por PHP</title>
</head>
<body>
<a href="destino.html">Esto es un link</a>
</body>
</html>
```



Links con variables

- El efecto del tag <a> ("anchor", o "ancla", que nos enlaza a otra página) es pedirle al servidor entregue la página especificada en el atributo "href" (es la abreviatura de "hiper-referencia").
- En nuestro caso del ejemplo la referencia es hacia la página "destino.html".
- Trabajando con páginas dinámicas, podemos aprovechar la posibilidad de "decirle algo" al servidor junto con el link, para que, además de pedirle ver una página en particular, nos deje ponerle valor a variables que luego estarán disponibles para que las use en esa misma página el intérprete de PHP.



Links con variables

- Supongamos que queremos definir una variable que se llame "nombre" y darle el valor "Pepe". Supongamos también que el link apunta a la página "recibe.php". El link que nos lleve a esa página debería ser así:
- `<a href="recibe.php?nombre=Pepe">Este es el link</a>`
- Luego de especificar la hiper-referencia (la página "recibe.php") se ha agregado un signo de pregunta. Este signo indica que a continuación **le vamos a enviar variables** al servidor web.

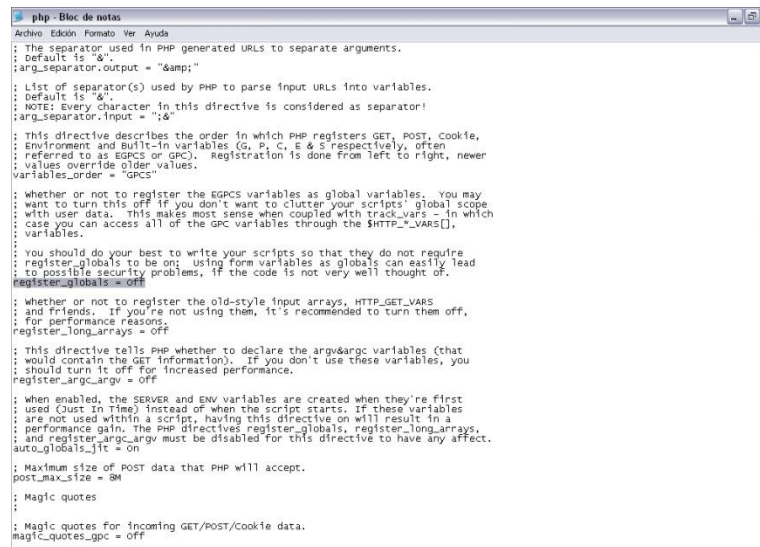


Links con variables

Importante:

No siempre se definen automáticamente las variables recibidas por parámetro en las páginas web, depende de una variable de configuración de PHP:

register_globals, tiene que estar activada para que así sea.



```
php - Bloc de notas
Archivo Edición Formato Ver Ayuda

; The separator used in PHP generated URLs to separate arguments.
; Default is '&'.
; arg_separator.output = "&";

; List of separator(s) used by PHP to parse input URLs into variables.
; Default is '&'.
; NOTE: Every character in this directive is considered as separator!
; arg_separator.input = "&";

; This directive describes the order in which PHP registers GET, POST, cookie,
; Environment and Built-in variables (G, P, C, & S respectively, often
; referred to as EGPCS or GPC). Registration is done from left to right, newer
; values override older values.
variables_order = "GPCS"

; Whether or not to register the EGPCS variables as global variables. You may
; want to turn this off if you don't want to clutter your scripts' global scope
; with user data. This makes most sense when coupled with track_vars in which
; case you can access all of the GPC variables through the $HTTP_*_VARS[]
; variables.
; You should do your best to write your scripts so that they do not require
; register_globals to be on; using form variables as globals can easily lead
; to possible security problems, if the code is not very well thought of.
register_globals = off

; Whether or not to register the old-style input arrays, HTTP_GET_VARS
; and friends. If you're not using them, it's recommended to turn them off,
; for performance reasons.
register_long_arrays = off

; This directive tells PHP whether to declare the argv&argc variables (that
; would contain the GET information). If you don't use these variables, you
; should turn it off for increased performance.
register_argv_argv = off

; When enabled, the SERVER and ENV variables are created when they're first
; used (Just in Time) instead of when the script starts. If these variables
; are not used within a script, having this directive on will result in a
; performance gain. The PHP directives register_globals, register_long_arrays,
; and register_argv_argv must be disabled for this directive to have any affect.
auto_globals_jit = on

; Maximum size of POST data that PHP will accept.
post_max_size = 8M

; Magic quotes
;
; Magic quotes for incoming GET/POST/Cookie data.
magic_quotes_gpc = off
```

php.ini cambiar **register_globals=off** por **register_globals=on**



Método GET y POST

GET y POST son dos métodos empleados para enviar los datos desde el navegador al servidor Web, generalmente utilizados al enviar formularios, especificados mediante la directiva *METHOD*.

La principal diferencia entre *POST* y *GET* es que *GET* envía los parámetros por medio de la URL del fichero que carguemos, mientras *POST* lo hace “oculto” del usuario, lo que lo hace más seguro al, por ejemplo, enviar información de un formulario.



Método GET

- Podemos experimentar el paso de variables directamente desde el navegador, escribiendo los nombres de las variables y sus valores en la barra de dirección del mismo; esto es muy práctico para chequear qué valores están llegando a la siguiente página y lo usaremos a menudo para detectar errores.
- `http://localhost/recibe.php?nombre=Pablo&apellido=Fernandez&edad=9`
- Cuando pulsemos "enter", esto saldrá enviado al servidor y surtirá el mismo efecto que los links que veníamos haciendo.
- A esta forma de pasar variables **en un link** se la conoce con el nombre de "método **GET**".



Variable \$_GET

- La variable \$_GET es otro vector asociativo que guarda toda la información enviada de una página a otra bajo el método “Get”.
- Por ejemplo, si nosotros tenemos un formulario con el valor “Get” en la propiedad “method”, que contenga los campos con nombre “Usuario” e “Email”, al enviar ese formulario podemos ver que nos dirige a la página especificada en la propiedad “action” con algunas variables en su url.



Variable \$_GET

- Cuando vemos que tras la url de un documento hay un signo de interrogación, lo que sigue es el nombre de una variable con su valor correspondiente (Variable=valor), y a cada variable a partir de la segunda en vez de un signo de interrogación se le antepondrá un ampersand (&).
- La variable \$_GET guarda cada una de estas variables, a las que se puede acceder usando el nombre de cada una de ellas como clave del array \$_GET.



Variable \$_Post

- \$_POST es otro vector asociativo que también guarda información transferida de una página a otra como la variable \$_GET, pero en este caso guarda la información transferida bajo otro método de formulario.
- La diferencia con \$_GET es que las variables enviadas bajo “post” no se verán en la url de la página a la que se envían los datos, por eso es más cómodo usar el método “post” en formularios en vez de “get”.



Formularios

- Otro método por excelencia para pasar valores entre dos páginas es el formulario.
- Se trata de un tag del lenguaje HTML, por lo que no es preciso que la página donde está el formulario lleve extensión ".php", puede ser un archivo ".html".
- Sus elementos principales son los siguientes:
 - El atributo "**action**", que indica "a qué página" le está pasando las variables, siendo la página que nos va a mostrar cuando pulsemos en el botón "Enviar", tal como si fuese un link hacia esa página.



Formularios

- El atributo "**method**", que especifica uno de los dos posibles "métodos" o formas de pasar las variables: **a la vista** de todos en la URL del navegador (method="**GET**") o si las vamos a pasar ocultas (method="**POST**").
- Algún **campo** que permita al usuario el ingreso de datos (**input**). Lo fundamental de cada campo será su **nombre** (atributo "name"), ya que ése será el nombre de la variable que enviará a la página de destino.
- Un **botón** para enviar los datos.
Esos son todos los elementos básicos de un formulario

